

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory work 5
Study the greedy algorithm design technique

Elaborated:
st. gr. FAF-242

Brînză Vasile

Verified:
asist. univ.

Fiștic Cristofor

Chișinău - 2026

TABLE OF CONTENTS

ALGORITHM ANALYSIS	3
Objective	3
Tasks	3
Theoretical Notes	3
Introduction	4
Comparison Metric	4
Input Format	4
IMPLEMENTATION	5
Prim's Algorithm	5
Kruskal's Algorithm	8
CONCLUSION	12
REFERENCES	14

ALGORITHM ANALYSIS

Objective

To study the greedy algorithm design technique.

Tasks

1. Study the greedy algorithm design technique;
2. To implement in a programming language algorithms Prim and Kruskal;
3. Empirical analyses of the Kruskal and Prim;
4. Increase the number of nodes in graph and analyze how this influences the algorithms.
Make a graphical presentation of the data obtained;
5. To make a report.

Theoretical Notes

Greedy algorithms represent a class of algorithmic techniques in which a problem is solved by making the locally optimal choice at each step, with the expectation that this sequence of choices leads to a globally optimal solution. At every stage, the algorithm selects the best available option without revisiting previous decisions. This often involves sorting data or using efficient data structures such as priority queues to quickly identify the next optimal candidate.

Although greedy algorithms are efficient and easy to implement, they do not always guarantee optimal results for all problems. Their correctness depends on specific properties of the problem, such as the greedy-choice property and optimal substructure. For example, problems like the 0/1 knapsack require dynamic programming, while others such as minimum spanning tree construction can be solved optimally using greedy strategies.

A key application of greedy algorithms is in graph theory, particularly in finding a Minimum Spanning Tree (MST). A spanning tree of a graph is a subgraph that connects all vertices using exactly $N-1$ edges without forming cycles. The minimum spanning tree is the spanning tree with the smallest possible total edge weight.

Prim's Algorithm constructs the MST by starting from an arbitrary node and iteratively adding the minimum weight edge that connects a visited node to an unvisited node. It maintains data structures to track selected vertices, minimum edge weights, and parent relationships. The algorithm expands the tree step by step, always choosing the closest vertex not yet included. This approach is particularly efficient for dense graphs, and its basic implementation has a time complexity of $O(N^2)$.

Kruskal's Algorithm, on the other hand, builds the MST by considering edges globally rather than growing from a single node. It begins by sorting all edges in increasing order of weight and

then selects edges one by one, ensuring that no cycles are formed. This process continues until $N-1$ edges have been included. Kruskal's algorithm typically uses a disjoint-set (Union-Find) structure for cycle detection and has a time complexity of $O(E \log N)$, making it efficient for sparse graphs.

Introduction

The minimum spanning tree problem is a fundamental problem in graph theory with practical applications in network design, infrastructure planning, and optimization of connected systems. It focuses on selecting a subset of edges that connects all vertices in a graph while minimizing the total cost.

This lab focuses on two well-known greedy algorithms: Prim's Algorithm and Kruskal's Algorithm. The goal is to study their design principles, implement them in a programming language, and evaluate their performance on graphs of increasing size.

The analysis includes examining how the number of nodes influences execution time and comparing the behavior of both algorithms under different conditions. The results are then visualized graphically to better understand their scalability and efficiency.

Comparison Metric

The comparison between the two algorithms is based primarily on execution time and graph size. Execution time measures how long each algorithm takes to compute the minimum spanning tree and is typically recorded in milliseconds. To ensure accuracy, measurements can be averaged over multiple runs.

Another important factor is the number of nodes and edges in the graph, which directly affects algorithm performance. As the graph grows larger, the efficiency of each approach becomes more evident. These metrics allow for a clear evaluation of how well each algorithm scales and under which conditions one outperforms the other.

Input Format

The algorithms are tested on randomly generated weighted graphs with varying numbers of nodes, such as 10, 50, 100, 200, and 500. For each graph, edges are assigned random weights within a fixed range, typically from 1 to 100.

Different graph configurations can be used to analyze performance. For instance, graphs may vary in size and number of edges to observe how increasing complexity impacts execution time. This setup helps illustrate the differences in behavior between Prim's Algorithm and Kruskal's Algorithm as the input grows.

IMPLEMENTATION

Prim's Algorithm

Prim's Algorithm is a greedy graph algorithm used to find the minimum spanning tree (MST) of a weighted graph. It builds the solution incrementally by starting from an arbitrary node and repeatedly adding the minimum weight edge that connects a node in the tree to a node outside the tree. At each step, it ensures that the partial solution remains a valid tree and gradually expands until all vertices are included.

Complexity Analysis

Time Complexity:

- Best Case: $O(N^2)$, when using a simple array-based implementation and minimal updates are required.
- Average Case: $O(N^2)$, as each vertex selection requires scanning all vertices.
- Worst Case: $O(N^2)$, particularly for dense graphs where many edges are considered.

(With a priority queue implementation, it can be improved to $O(E \log N)$).

Auxiliary Space: $O(N)$, required for storing arrays such as visited nodes, minimum edge values, and parent relationships.

Algorithm Description

Prim's Algorithm works by:

- Initialize: Select an arbitrary starting node. Set its value to 0 and all other nodes to infinity. Mark all nodes as not included in the MST.
- Select: Choose the node with the smallest value that is not yet included in the MST.
- Update: For all adjacent nodes, if the edge weight is smaller than their current value, update their value and parent.
- Repeat: Continue selecting and updating until all nodes are included in the MST.

Implementation:

```
prim.py > prim
1  import heapq
2
3  def prim(graph, start=0):
4      n = len(graph)
5      visited = [False] * n
6      min_heap = [(0, start, -1)] # (weight, node, parent)
7
8      mst_weight = 0
9      mst_edges = []
10
11     while min_heap:
12         weight, node, parent = heapq.heappop(min_heap)
13
14         if visited[node]:
15             continue
16
17         visited[node] = True
18         mst_weight += weight
19
20         if parent != -1:
21             mst_edges.append((parent, node, weight))
22
23         for neighbor, w in graph[node]:
24             if not visited[neighbor]:
25                 heapq.heappush(min_heap, (w, neighbor, node))
26
27     return mst_weight, mst_edges
28
29
```

Figure 1: Prim's algorithm in Python

Prim's Algorithm was implemented in Python using an adjacency list representation of the graph. Three arrays were used: one to track whether a node is included in the MST, one to store the minimum edge weight required to connect each node, and one to maintain the parent structure of the tree.

At each step, the algorithm selects the node with the smallest connection cost that is not yet included and updates its neighbors accordingly. This approach ensures that the minimum spanning tree is constructed incrementally while maintaining optimal choices at every step.

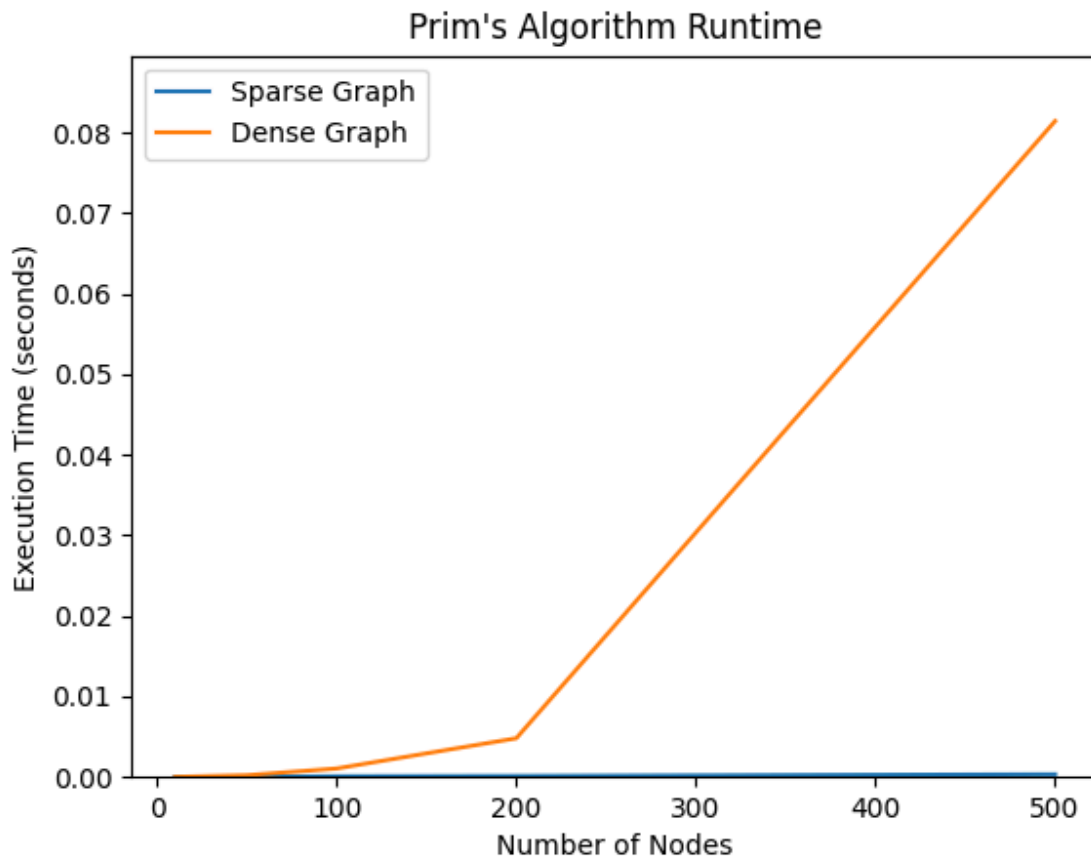
Results:

```
=== Prim's Algorithm Results ===  
Nodes: 10 | Sparse: 0.000011s | Dense: 0.000007s  
Nodes: 50 | Sparse: 0.000024s | Dense: 0.000206s  
Nodes: 100 | Sparse: 0.000084s | Dense: 0.001045s  
Nodes: 200 | Sparse: 0.000118s | Dense: 0.004802s  
Nodes: 500 | Sparse: 0.000325s | Dense: 0.081455s
```

Figure 2: Results of Prim's algorithm for inputs

The algorithm was tested on randomly generated graphs with node counts of 10, 50, 100, 200, and 500. For each size, graphs were generated with varying edge densities and random weights between 1 and 100. The execution time was recorded for each configuration and averaged over multiple runs.

Graph plot:



The results show that the execution time increases as the number of nodes grows. For smaller graphs, the runtime increase is minimal, but as the graph becomes larger, the growth becomes more noticeable due to the $O(N^2)$ complexity.

For dense graphs, the algorithm performs relatively efficiently compared to alternatives, since it does not require sorting all edges. However, as the number of nodes increases significantly, the cost of repeatedly selecting the minimum value node dominates the runtime.

This behavior confirms that Prim's Algorithm is suitable for dense graphs, but its performance degrades with large input sizes when implemented without advanced data structures.

Kruskal's Algorithm

Kruskal's algorithm is a greedy graph algorithm used to find the minimum spanning tree (MST) of a weighted graph. Unlike Prim's approach, it builds the solution by selecting edges globally rather than expanding from a single node. At each step, the algorithm chooses the smallest available edge that does not form a cycle, ensuring that the resulting structure remains a valid tree.

Complexity Analysis

Time Complexity:

- Best Case: $O(E \log N)$, when the graph is sparse and sorting dominates the computation.
- Average Case: $O(E \log N)$, as sorting edges is required regardless of structure.
- Worst Case: $O(E \log N)$, when all edges must be processed and cycle checks are performed.

Auxiliary Space: $O(N)$, required for storing the disjoint-set (Union-Find) structure used for cycle detection.

Algorithm Description

Kruskal works by:

- Initialize: Sort all edges of the graph in increasing order of weight. Initialize a disjoint-set structure where each node is its own component.
- Select: Pick the smallest edge from the sorted list.
- Check: Determine whether adding this edge forms a cycle using the disjoint-set structure.
- Union: If no cycle is formed, include the edge in the MST and merge the two components.

- Repeat: Continue until $N-1$ edges have been selected.

Implementation:

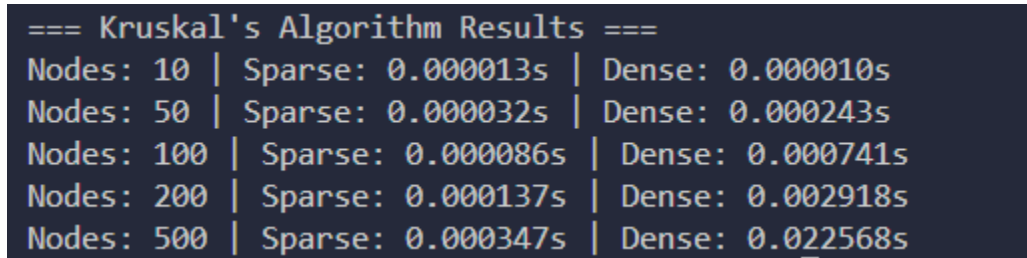
```
kruskal.py > ...
1  def find(parent, x):
2      if parent[x] != x:
3          parent[x] = find(parent, parent[x]) # Path compression
4      return parent[x]
5
6  def union(parent, rank, x, y):
7      root_x = find(parent, x)
8      root_y = find(parent, y)
9
10     if root_x != root_y:
11         if rank[root_x] < rank[root_y]:
12             parent[root_x] = root_y
13         elif rank[root_x] > rank[root_y]:
14             parent[root_y] = root_x
15         else:
16             parent[root_y] = root_x
17             rank[root_x] += 1
18
19
20  def kruskal(n, edges):
21      edges.sort(key=lambda x: x[2]) # sort by weight
22
23      parent = list(range(n))
24      rank = [0] * n
25
26      mst_weight = 0
27      mst_edges = []
28
29      for u, v, w in edges:
30          if find(parent, u) != find(parent, v):
31              union(parent, rank, u, v)
32              mst_edges.append((u, v, w))
33              mst_weight += w
34
35      return mst_weight, mst_edges
36
```

Figure 4: Kruskal's in Python

Kruskal's algorithm was implemented in Python using an edge list representation of the graph. All edges are first sorted based on their weights. A disjoint-set (Union-Find) data structure is used to efficiently detect cycles during edge selection.

At each step, the algorithm processes the next smallest edge and checks whether it connects two different components. If it does, the edge is added to the minimum spanning tree, and the components are merged. This ensures that no cycles are formed while maintaining the minimal total weight.

Results:



```
=== Kruskal's Algorithm Results ===
Nodes: 10 | Sparse: 0.000013s | Dense: 0.000010s
Nodes: 50 | Sparse: 0.000032s | Dense: 0.000243s
Nodes: 100 | Sparse: 0.000086s | Dense: 0.000741s
Nodes: 200 | Sparse: 0.000137s | Dense: 0.002918s
Nodes: 500 | Sparse: 0.000347s | Dense: 0.022568s
```

Nodes	Sparse	Dense
10	0.000013s	0.000010s
50	0.000032s	0.000243s
100	0.000086s	0.000741s
200	0.000137s	0.002918s
500	0.000347s	0.022568s

Figure 5: Kruskal's runtime

The algorithm was tested on randomly generated graphs with node counts of 10, 50, 100, 200, and 500. For each size, both sparse and dense graphs were generated, with edge weights assigned randomly between 1 and 100. Execution times were recorded and averaged over multiple runs.

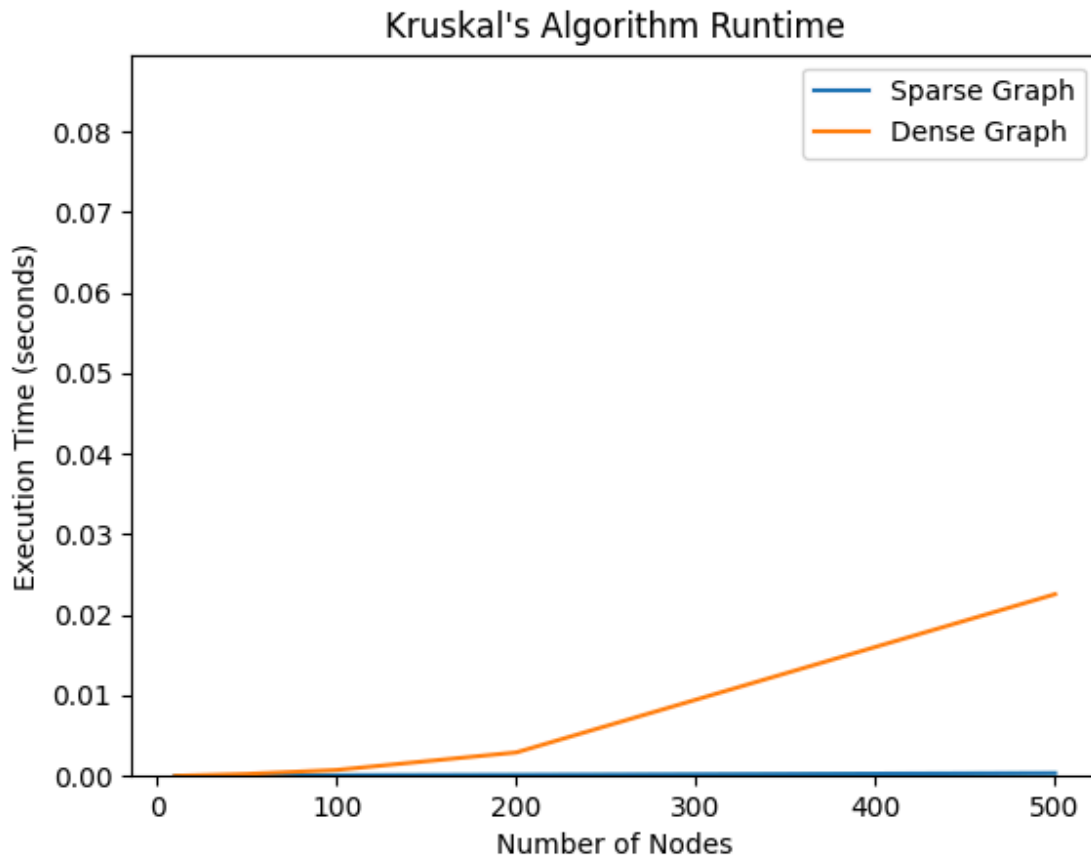


Figure 6: Graph of Kruskal's Runtime

The results indicate that execution time increases with the number of edges rather than just the number of nodes. For sparse graphs, where the number of edges is relatively small, the algorithm performs efficiently. However, for dense graphs, the sorting step becomes costly, leading to a noticeable increase in runtime.

This confirms that Kruskal's Algorithm is particularly well-suited for sparse graphs, while its performance is affected more significantly by graph density compared to node count alone.

CONCLUSION

In conclusion, Prim's Algorithm and Kruskal's Algorithm solve the minimum spanning tree problem using the same greedy principle, but follow fundamentally different strategies. Prim's Algorithm builds the solution incrementally by expanding from a starting node, always selecting the closest vertex to the existing tree. This makes it particularly effective when the graph is dense, as it avoids processing all edges globally and instead focuses only on relevant neighboring connections.

Kruskal's Algorithm, in contrast, takes a global perspective by sorting all edges and selecting the smallest ones while avoiding cycles. This approach is well suited for sparse graphs, where the number of edges is relatively small and sorting does not introduce significant overhead. Its reliance on efficient cycle detection through disjoint-set structures ensures correctness while maintaining good performance.

However, these differences lead to distinct performance trade-offs. Prim's Algorithm does not require sorting all edges and therefore scales better with increasing edge density, but its performance depends on how efficiently the next minimum vertex is selected. Without optimized data structures, its quadratic complexity becomes noticeable for large graphs. Kruskal's Algorithm, while efficient for sparse graphs, becomes less practical for dense graphs due to the cost of sorting a large number of edges.

The experimental results support these observations. Prim's Algorithm demonstrates stable and efficient behavior as graph density increases, while Kruskal's Algorithm performs best when the number of edges remains low. As the number of nodes and edges grows, Kruskal's runtime is increasingly dominated by the sorting step, whereas Prim's runtime is influenced more by the number of vertices.

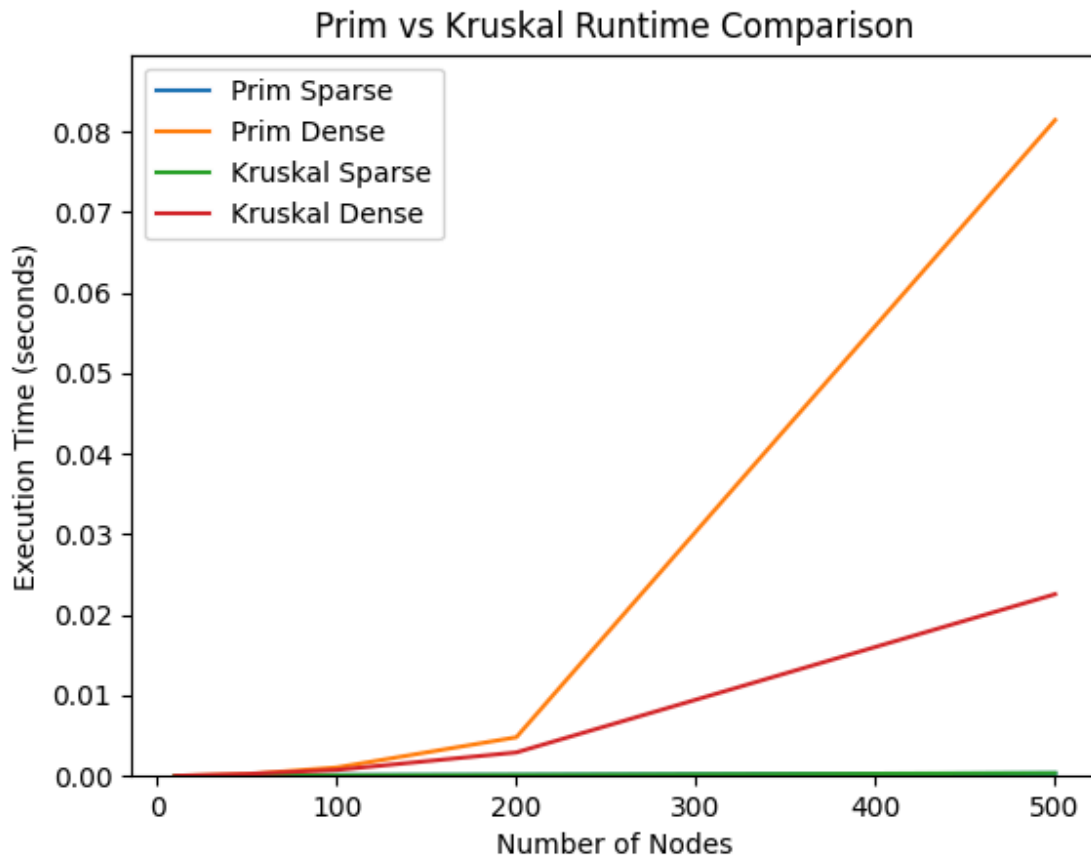


Figure 7: Comparative runtime analysis of Prim's and Kruskal's algorithms on graphs of increasing size

Overall, the choice between the two algorithms should depend on the structure of the graph. For dense graphs or scenarios where adjacency-based expansion is more efficient, Prim's Algorithm is generally the better option. For sparse graphs or cases where edge-based processing is simpler and more natural, Kruskal's Algorithm provides a more efficient solution.

REFERENCES

1. <https://github.com/kynexi/AA-labs/tree/main/Lab5>
2. <https://www.geeksforgeeks.org/greedy-algorithms/>
3. https://www.tutorialspoint.com/data_structures_algorithms/greedy_algorithms.htm
4. <https://www.scaler.com/topics/prims-and-kruskal-algorithm/>